

SENAC · LÓGICA DE PROGRAMAÇÃO

Lista de Exercícios Lógica de Programação

Linguagem C#

27 exercícios · 6 módulos

Do "Olá, Mundo!" aos primeiros sistemas completos

Aluno(a): _____

Turma: _____

Orientações gerais

- Resolva os exercícios **na ordem**: cada módulo usa o conteúdo do anterior.
- Crie um projeto de console em C# para cada exercício (`dotnet new console`) ou reaproveite um único projeto trocando o conteúdo do `Main` .
- Sempre **teste com valores diferentes**, inclusive valores inválidos (zero, negativos, texto onde se espera número).
- Leitura do teclado: `Console.ReadLine()` . Escrita na tela: `Console.WriteLine()` .
- Números decimais dependem da configuração regional: no Brasil normalmente usa-se **vírgula** (ex.: `1,75`).

ANTES DE COMEÇAR

Anatomia de um programa em C#

Todo exercício desta lista tem o mesmo esqueleto. Entenda cada linha dele uma única vez e você não precisa mais pensar nisso:

```
using System;                                // 1. importa o namespace System

class Program                                // 2. o código mora dentro de uma classe
{
    static void Main()                        // 3. o programa COMEÇA aqui
    {
        Console.WriteLine("Olá Mundo!");    // 4. escreve na tela
    }
}
```

1. using System;

Em C#, cada classe pronta da biblioteca padrão mora dentro de um **namespace** — um "sobrenome" que evita conflito de nomes. A classe `Console` mora no namespace `System`, ou seja, o nome completo dela é `System.Console`.

Sem o `using`, você teria que escrever o endereço completo toda vez:

`System.Console.WriteLine("Olá");`. O `using System;` é um atalho que avisa o compilador: *"quando eu escrever um nome solto, procure também dentro de System"*. A partir daí, `Console` sozinho já basta.

Isso vale para tudo que mora em `System`: `Math`, `Random`, `Convert`, `DateTime`. É parecido com o `import` do Python — mas atenção: ele **não copia código nenhum** para o seu arquivo, apenas ensina o compilador a resolver nomes curtos.

Cuidado: é `System` com **S maiúsculo**. C# diferencia maiúsculas de minúsculas, então `using system;` não compila.

2. class Program / 3. static void Main()

Em C# não existe código solto: tudo precisa estar dentro de uma **classe** (aqui chamada `Program`, mas o nome é livre). E, dentro dela, o método `Main` tem um papel especial — é o **ponto de partida**. Quando você roda o programa, o computador procura o `Main` e executa as linhas dele, de cima para baixo.

Nos Módulos 1 a 4, todo o seu código vai dentro das chaves do `Main`. A partir do Módulo 5 você vai criar *outros* métodos ao lado dele.

4. Console.WriteLine(...)

Leia da esquerda para a direita — são três coisas encaixadas:

Parte	O que é
<code>Console</code>	a classe que representa o terminal (a tela e o teclado)
<code>.</code>	o operador de acesso: "de dentro de"
<code>WriteLine()</code>	o método que escreve na tela; o que vai entre parênteses é o dado que você entrega a ele

O `Line` no final do nome é literal: ele escreve **e quebra a linha**. O irmão dele é o `Console.Write`, idêntico, mas **sem** a quebra de linha. Por isso usamos `Write` nas perguntas e `WriteLine` nas respostas:

```
Console.Write("Digite seu nome: "); // cursor fica na MESMA linha
string nome = Console.ReadLine(); // o usuário digita ao lado

Console.WriteLine($"Olá {nome}!"); // escreve e pula linha
Console.WriteLine("Tudo bem?"); // sai numa linha nova
```

Resultado na tela:

```
Digite seu nome: Pablo
Olá Pablo!
Tudo bem?
```

5. Lendo do teclado: Console.ReadLine()

O `Console.ReadLine()` devolve **sempre um texto** (`string`) — mesmo quando o usuário digita "30". Para fazer conta, é preciso **converter**:

Quero ler	Escrevo
Um texto (nome)	<code>string nome = Console.ReadLine();</code>
Um número inteiro (idade)	<code>int idade = int.Parse(Console.ReadLine());</code>
Um número decimal (altura)	<code>double altura = double.Parse(Console.ReadLine());</code>

O `Parse` é justamente quem traduz o texto `"30"` no número `30`.

6. O símbolo \$ (interpolação de string)

Colocando um `$` antes das aspas, tudo que estiver entre `{ }` dentro do texto é substituído pelo **valor** da variável:

```
int idade = 30;  
Console.WriteLine($"Você possui {idade} anos."); // → Você possui 30 anos.
```

Sem o `$`, o texto sairia literalmente como `Você possui {idade} anos.`

⚠ Vírgula ou ponto nos números decimais?

Depende da configuração regional do computador. Em português do Brasil, digite **vírgula**: `1,75`. Em inglês, digite **ponto**: `1.75`.

Isso importa mais do que parece: numa máquina configurada em inglês, digitar `1,75` **não dá erro** — o programa entende `175` e o resultado sai errado silenciosamente. Se seus cálculos derem valores absurdos, é quase sempre isso. Teste antes com um valor simples para descobrir qual separador a sua máquina espera.

MÓDULO 1

Fundamentos

Entrada de dados, saída de dados, variáveis e operadores aritméticos.

01 Olá, Mundo

Objetivo: saída de dados.

Exiba a mensagem:

```
Olá Mundo!
```

Conceitos: Saída

02 Apresentação

Solicite ao usuário:

- Nome
- Idade

Mostre:

```
Olá Pablo!  
Você possui 30 anos.
```

Conceitos: Entrada Saída Variáveis

03 Calculadora Simples

Solicite dois números e mostre o resultado de:

- Soma
- Subtração
- Multiplicação
- Divisão

Conceitos: Operadores aritméticos Conversão de tipos

04 Área do Retângulo

Solicite:

- Base
- Altura

Calcule e exiba a área.

$$\text{Área} = \text{Base} \times \text{Altura}$$

05 Conversor de Temperatura

Converta uma temperatura de Celsius para Fahrenheit.

$$F = (C \times 9 / 5) + 32$$

Atenção: cuidado com a divisão inteira. Use tipos decimais.

MÓDULO 2

Decisão

Estruturas condicionais: `if`, `else if`, `else` e operadores lógicos.

06 Número Positivo

Leia um número e informe se ele é:

- Positivo
- Negativo
- Zero

07 Maior Número

Leia três números e mostre qual é o maior.

Conceitos: `if aninhado` `Operadores lógicos`

08 Aprovação Escolar

Leia a Nota 1 e a Nota 2, calcule a média e informe o resultado:

Média	Resultado
Maior ou igual a 7	Aprovado
De 5 a 6,9	Recuperação
Menor que 5	Reprovado

09 Ano Bissexto

Leia um ano e informe se ele é bissexto.

Regra: é bissexto o ano divisível por 4, *exceto* os divisíveis por 100 — a menos que também sejam divisíveis por 400.

Conceitos: `Operador módulo (%)` `E / OU lógico`

10 Calculadora de IMC

Leia o peso (kg) e a altura (m), calcule o IMC e informe a classificação.

$$\text{IMC} = \text{Peso} / (\text{Altura} \times \text{Altura})$$

IMC	Classificação
Abaixo de 18,5	Abaixo do peso
18,5 a 24,9	Peso normal
25,0 a 29,9	Sobrepeso
30,0 ou mais	Obesidade

MÓDULO 3

Repetição

Laços `for`, `while` e `do...while`; acumuladores e contadores.

11 Contador

Exiba os números de 1 até 100:

```
1
2
3
...
100
```

12 Tabuada

Leia um número e exiba sua tabuada de 1 a 10.

```
Tabuada do 7:
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

13 Soma dos Números

Leia números repetidamente até que o usuário digite **0**. Ao final, mostre a soma de todos os números digitados.

Conceitos: Laço com condição de parada Acumulador

14 Adivinhe o Número

O computador escolhe um número aleatório entre 1 e 100. O usuário tenta acertar. A cada tentativa, informe:

- **Maior** — o número secreto é maior que o palpite
- **Menor** — o número secreto é menor que o palpite
- **Acertou!** — fim do jogo

Ao final, mostre em quantas tentativas o usuário acertou.

15 Fatorial

Leia um número e calcule o seu fatorial.

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Lembre-se: **0! = 1**.

MÓDULO 4

Vetores

Arrays: declaração, preenchimento, percurso e índices.

16 Média da Turma

Leia as notas de 10 alunos e mostre:

- Média da turma
- Maior nota
- Menor nota

17 Encontrar Maior Valor

Leia 20 números e mostre o maior deles.

Desafio extra: mostre também em qual posição do vetor ele está.

18 Inverter Vetor

Leia 10 números e exiba-os em ordem inversa (do último para o primeiro).

Conceitos: Percurso reverso Índices

19 Contagem

Leia 20 números e informe:

- Quantos são pares
- Quantos são ímpares

MÓDULO 5

Funções

Métodos: parâmetros, retorno e reaproveitamento de código.

20 Função Soma

Crie uma função que receba dois valores e retorne a soma:

```
Somar(a, b)
```

Chame-a a partir do programa principal e exiba o resultado.

21 Calculadora com Funções

Reescreva a calculadora do Exercício 3 usando quatro funções:

- Somar
- Subtrair
- Multiplicar
- Dividir

Trate a divisão por zero.

22 Verificador de Número Primo

Crie uma função que receba um número e retorne **verdadeiro** ou **falso**, indicando se ele é primo.

Lembrete: número primo é divisível apenas por 1 e por ele mesmo. O 1 **não** é primo.

23 Palíndromo

Crie uma função que verifica se uma palavra é um palíndromo (lê-se igual de trás para frente).

```
arara → é palíndromo  
senac → não é palíndromo
```

Desafio extra: ignore maiúsculas/minúsculas e espaços.

MÓDULO 6

Problemas Reais

Juntando tudo: laços, decisões, vetores e funções em programas completos.

24 Caixa Eletrônico

O usuário informa um valor de saque. O programa informa a **menor quantidade possível de notas**.

Notas disponíveis: **100, 50, 20, 10, 5, 2 e 1**.

EXEMPLO DE SAÍDA PARA O VALOR 286

```
2 notas de 100  
1 nota de 50  
1 nota de 20  
1 nota de 10  
1 nota de 5  
1 nota de 1
```

25 Sistema de Mercado

Permita cadastrar vários produtos, informando:

- Nome
- Preço
- Quantidade

Ao final, mostre a lista de produtos e o **total da compra**.

26 Sistema de Login

O sistema possui um único usuário válido:

Usuário	Senha
admin	123456

Permita até **três tentativas**. Se errar as três, bloqueie o acesso.

27 Agenda Telefônica

Crie uma agenda que armazene:

- Nome
- Telefone

O programa deve exibir um menu com as opções:

- **Inserir** um novo contato
- **Listar** todos os contatos
- **Procurar** um contato pelo nome
- **Sair**

O menu deve repetir até o usuário escolher "Sair".